Git Guild 架构辩论赛记录

文档说明

本文件整理了围绕 Git Guild 架构选型进行的多轮 AI 问答记录。为便于阅读，统一采用以下结构：

- 每轮包含一个 `Prompt` 和对应的 `AI 回答`
- 保留原始问题与核心回答内容
- 通过标题、分段和表格提升可读性

说明：本文中的 `AI 回答` 保留了不同 Prompt 下 AI 的原始建议。这些内容用于体现 AI 建议差异，不代表团队最终选型。团队最终结论以文末“当前结论”和《架构设计文档》为准。

第一轮：开放式架构咨询

Prompt

> 我正在开发一个面向开源初学者的游戏化代码托管与悬赏协作平台，功能包括仓库托管、Issue/PR 工作流、任务发布与接取、XP/等级/徽章系统。推荐什么架构风格？

AI 回答

这是一个非常有趣且具有挑战性的项目。它同时结合了代码托管与游戏化系统，因此架构既要能处理 Git 相关的高一致性、高 IO 操作，也要能支持经验值、勋章和动态反馈这类高频触发业务。

针对这一需求，AI 推荐采用：

- 事件驱动的微服务架构（Event-Driven Microservices）
- 在单个服务内部结合整洁架构（Clean Architecture）

1. 为什么选择微服务

- 关注点分离：代码托管逻辑与游戏化逻辑属于不同领域，前者强调稳定性，后者强调灵活性与规则变化。
- 独立扩展：任务系统、XP 计算、勋章结算等负载模式不同，微服务便于分别扩容。

2. 为什么强调事件驱动

在该平台中，很多操作都会触发一系列连锁反应。示例如下：

1. 用户提交一个 PR。
2. 系统发布 `PR_SUBMITTED` 事件到消息队列。
3. 不同服务异步处理：
   - XP 服务给用户增加经验值
   - 勋章服务检查是否达成成就条件
   - 动态服务发布平台广播

3. 关键模块拆分建议

模块名称	职责描述	技术要点
Git Core Service	处理 `git push/pull`、分支管理、文件差异对比	可封装 Git 二进制或使用 Libgit2，需要强一致性
Workflow Service	管理 Issue 状态、PR 审查流、代码评论	使用状态机驱动，保证流程合规
Bounty/Task Service	管理任务发布、接取、保证金锁定与结算	涉及积分或虚拟货币，需要严格事务
Gamification Engine	负责 XP、等级曲线和勋章逻辑	可使用规则引擎或策略模式
Notification Service	推送 XP 获取提醒、PR 被合并提醒	可基于 WebSocket 或 SSE

4. 单个服务内部结构建议

建议在每个服务内部采用整洁架构或六边形架构：

- 领域层（Domain）：定义 `User`、`Repository`、`Badge`、`XP` 等核心实体与规则
- 用例层（Use Cases）：实现“领取悬赏任务”等业务流程
- 接口适配层（Adapters）：负责 REST API、数据库、缓存等外部接口的适配

这样可以避免业务逻辑与具体框架或数据库强耦合。

5. 针对初学者平台的额外建议

1. 沙盒化与安全性：在容器中运行有风险的代码执行过程。
2. 低延迟反馈：可用 Redis 缓存 XP、等级和排行榜。
3. Webhook 集成：如果前期不想自研 Git 托管能力，可先接入 GitHub 或其他现有平台的 Webhook。

6. 技术栈建议

- 后端：Golang 或 Node.js
- 通信：gRPC + RabbitMQ/NATS
- 存储：PostgreSQL + Redis + MinIO

第二轮：加入课程项目约束后的架构咨询

Prompt

> 项目：Git Guild，面向开源初学者的游戏化代码托管与悬赏协作平台。核心功能包括仓库托管、Issue/PR 流程、任务悬赏板、推荐匹配、XP/等级/徽章成长体系、排行榜。
>
> 约束条件：
>
> - 团队 4 人，学生课程项目，周期 10 周，每周人均约 6 小时
> - 团队中有成员有 Java 后端开发经验
> - 需要覆盖推荐系统这一核心功能，并为导师模式等后续能力预留演进空间
> - 平台需要同时服务两类用户：接任务的初学者和发任务的项目维护者
>
> 基于以上背景，推荐什么架构风格？请说明理由，并给出模块划分建议。

AI 回答

在加入团队规模、周期和成员技术背景后，AI 将推荐方案调整为：

- 模块化单体架构（Modular Monolith）

1. 推荐理由

- 开发效率更高：10 周课程项目中，微服务会带来额外的跨服务调用、分布式事务和环境部署成本。
- 技术栈匹配：团队已有 Java 后端经验，Spring Boot 可以快速支撑原型开发。
- 易于演进：通过按功能模块划分包结构，可以先将推荐系统作为核心模块纳入单体实现，后续再按需要把推荐或导师模块逐步拆分出去。

2. 模块划分建议

建议划分为 6 个核心业务模块和 1 个基础支撑模块：

协同空间模块（Workspace Module）

- 目标用户：初学者与维护者
- 职责：仓库托管元数据管理、Issue 开启/关闭、PR 提交、版本关联

悬赏大厅模块（Bounty Board Module）

- 目标用户：维护者发布任务，初学者接取任务
- 职责：任务发布、押金托管、任务认领、验收流程、奖励结算

成长体系模块（Gamification Module）

- 目标用户：初学者
- 职责：XP 计算、等级提升、徽章授予、排行榜聚合
- 解耦方式：监听领域事件，例如 `TaskCompletedEvent`

推荐匹配模块（Recommendation Module）

- 目标用户：主要面向初学者，也服务于维护者进行任务曝光
- 职责：基于技术栈标签、任务历史、参与行为和成长阶段进行任务推荐与排序
- 实现方式：作为核心模块直接纳入单体架构，并与任务、用户、成长模块共享领域数据

社交与激励模块（Social Module）

- 目标用户：双方
- 职责：评论系统、导师/学徒关系预留、关注动态

用户中心模块（User/Auth Module）

- 职责：角色权限控制、第三方 OAuth 登录

3. 推荐技术栈与演进路径

维度	推荐选型	备注
后端	Spring Boot 3 + JPA/MyBatis Plus	团队有 Java 经验，上手快
数据库	PostgreSQL / MySQL	适合任务状态与积分结算
缓存	Redis	适合排行榜和 XP 高频读取
扩展点	Spring Event	通过应用内事件解耦模块

4. 核心模块与后续演进建议

- 推荐系统：作为核心业务模块纳入当前整体架构，基于任务历史、技术栈标签和 XP 特征推荐任务

- 导师模式：作为后续能力在 Social Module 上扩展辅导与激励功能

5. AI 给出的实施建议

1. 前 4 周优先打通“托管-接榜-完成-奖励”的闭环。
2. 前后端通过 RESTful API 严格交互。
3. XP 结算逻辑应尽早独立设计，避免与仓库逻辑混在一起。

第三轮：候选架构方案对比

Prompt

> 为 Git Guild（游戏化代码托管 + 悬赏协作平台，4 人学生团队，10 周课程项目）选择架构风格。请对比以下至少 2 种候选方案（如分层架构、微服务架构、MVC 扩展架构）：
>
> 对比维度：
>
> 1. 开发复杂度
> 2. 可维护性
> 3. 可扩展性
> 4. 性能
> 5. 团队学习成本
> 6. 10 周内可行性
>
> 给出明确的推荐结论，对比多种方案的优劣，并说明如果团队中有 Java 后端经验和前端开发经验，对选择是否有影响。

AI 回答

AI 对三种候选架构进行了对比：

对比维度	MVC 扩展架构（单体）	分层架构（Layered）	微服务架构（Microservices）
开发复杂度	低，单一代码库，调试部署直观	中，逻辑清晰但仍需结构设计	极高，需要处理服务发现和分布式问题
可维护性	中，后期可能演变为大泥球	高，各层职责明确	极高，但运维压力很大
可扩展性	低，只能整体扩容	中，便于后续抽取模块	极高，可按服务独立扩容
性能	高，进程内调用无网络开销	高，与单体接近	中到低，存在网络与序列化损耗

对比维度	MVC 扩展架构（单体）	分层架构（Layered）	微服务架构（Microservices）
团队学习成本	极低	低	极高
10 周内可行性	极高	极高	极低

AI 的推荐结论

AI 强烈推荐：

- 分层架构（Layered Architecture）
- 实际落地形式为前后端分离的模块化单体

推荐理由

1. 10 周周期下，交付速度与复杂度控制是核心矛盾。
2. 通过 `Web 层 - Service 层 - Domain 层 - Data 层` 的分层设计，可以获得接近微服务的解耦效果，同时避免分布式系统的高复杂度。
3. 4 人团队中，前后端可以围绕接口契约并行开发。

团队经验对选型的影响

Java 后端经验

- Spring Boot 非常适合实现分层架构
- JPA 或 MyBatis 提供成熟的数据访问模式
- Java 的接口与分层约束有利于多人协作

前端开发经验

- 架构会自然演变为前后端分离模式
- 前端可使用 React 或 Vue 构建 SPA
- 前后端可基于 Mock 数据和 REST API 并行推进

额外建议

虽然整体采用单体分层架构，但可以在内部加入轻量级事件机制：

- 例如在“代码提交完成”后异步更新 XP
- 这样能让游戏化逻辑与核心流程保持解耦

AI 给出的阶段建议

- Week 1-2：环境搭建与分层模型定义
- Week 3-8：核心业务开发
- Week 9-10：测试、修复和演示准备

第四轮：以架构师身份进行论证与风险提醒

Prompt

> 请你扮演一位资深系统架构师，为教育类课程项目提供架构咨询。
>
> 项目背景：Git Guild，面向开源初学者的游戏化代码托管与悬赏协作平台。功能涵盖仓库托管、Issue/PR 工作流、任务发布与接取、推荐匹配、XP/徽章/等级成长体系。团队为 4 人学生小组，10 周交付周期，每周人均 6 小时。团队技术背景：有成员有 Java 后端经验，有成员会一些前端开发技术，但技术栈尚未最终确定。后续可能扩展导师匹配功能。
>
> 请以架构师身份，分析最适合的架构风格，论证合理性，并指出学生团队常见架构选型误区（如过度设计、技术栈先行）可能带来的风险。

AI 回答

AI 从“课程项目的工程现实”角度进行了论证，核心判断是：

- 在总人工时约 240 小时的背景下，架构优先级应是交付速度与认知负担，而不是高并发或高可用
- 因此最合适的方案仍然是模块化单体架构（Modular Monolith）

1. 架构建议

推荐采用：

- 单体优先，逻辑解耦
- 经典 `Controller - Service - Repository` 分层模式
- 按功能模块进行严格分包，例如：
  - `com.gitguild.repo`
  - `com.gitguild.gamification`
  - `com.gitguild.bounty`
  - `com.gitguild.recommendation`

同时建议引入轻量级事件机制：

- 例如在 PR 合并后发布 `PRMergedEvent`

- 由 XP 或徽章模块异步处理后续逻辑

2. 技术栈建议

维度	推荐选型	论证理由
后端	Spring Boot（Java）	团队已有 Java 经验，生态成熟
前端	React + Tailwind CSS	前端搭建速度快
数据库	PostgreSQL	可满足结构化数据需求
中间件	Redis	用于排行榜和缓存
Git 集成	GitLab/GitHub API 或 Gitea API	不建议自研 Git 存储底层

3. 学生团队常见误区与风险

误区一：技术栈先行

现象：

- 为了学习新工具而选择不熟悉的语言或框架
- 在不必要时引入 Kafka、Elasticsearch 等复杂组件

风险：

- 10 周项目中，学习曲线本身就是重大风险
- 如果第 4 周仍在解决环境或语法问题，项目很可能无法完整交付

建议：

- 使用团队最熟悉的技术完成最核心的业务问题

误区二：过度设计

现象：

- 为推荐系统这一核心模块预先设计过度复杂的微服务接口
- 为不存在的高并发场景提前设计分库分表

风险：

- 当前瓶颈不是性能，而是功能完成度
- 复杂架构会产生大量样板代码，拖慢实现速度

建议：

- 遵循 YAGNI 原则
- 优先保证数据模型清晰，扩展留到后续迭代

误区三：忽略可观测性与部署

现象：

- 本地能运行，但部署后排错困难
- 缺少自动化部署和基础日志

风险：

- 最后两周容易陷入环境与 Bug 排查泥潭

建议：

- 第一周就搭建好基础部署和日志机制
- 尽早建立 CI/CD 流程

4. AI 给出的起步建议

1. 先明确核心领域模型，如 `User`、`Task`、`Repository`、`Reward`。
2. 前后端先约定 API 契约，再并行开发。
3. 第一周尽量打通最小可运行闭环，例如：登录 -> 创建 Issue -> 增加 XP。

讨论结果整理

通过多轮提问可以看出，AI 的建议经历了从“理想化架构”到“受课程约束后的务实架构”的收敛过程：

- 在不设约束时，AI 更倾向于事件驱动微服务
- 在加入团队规模、时间和技术背景后，AI 明确收敛到模块化单体架构
- 在对比候选方案后，分层单体在开发复杂度、学习成本和 10 周可行性上优势明显
- 在架构师视角下，AI 进一步强调了学生项目中常见的过度设计风险

当前结论

结合项目约束与课程目标，Git Guild 更适合采用：

- 前后端分离
- 后端使用模块化单体架构
- 内部采用分层设计

- 平台自有账号体系
- 集成 Gitea 作为外部代码托管底座
- 在模块之间引入轻量级事件机制

该方案既能承载当前课程项目中的推荐系统、任务协作与成长体系等核心能力，也为导师模式等后续功能保留了演进空间。